



LLMs - In-Context Learning, RL & Evaluation

Benjamin Piwowarski

Lecture 4

<https://master-mind.isir.upmc.fr/llm/slides>

Press ? for help on keyboard shortcuts

[Slides in PDF](#)



Machine Learning &
Deep Learning for
Information Access

TODAY'S LECTURE

1. In-Context Learning & Prompting

- Prompting basics and types
- Zero-shot vs few-shot learning
- Chain-of-Thought and advanced prompting

2. Reinforcement Learning for LLMs

- RLHF pipeline
- PPO, DPO, GRPO
- Towards LLM agents (reasoning + tools)

3. Evaluation & Dataset Generation

- Evaluating LLMs (perplexity, metrics)
- LLMs as judges
- Synthetic data generation
- Contamination concerns

IN-CONTEXT LEARNING & PROMPTING

IN-CONTEXT LEARNING & PROMPTING

PROMPTING

KEY PROPERTIES OF LLMs

- Pre-trained on diverse texts and tasks
- Multiple knowledge sources embedded in parameters
- Traditional ML: same distribution in train and inference
- **LLMs**: inference can be out-of-distribution through prompt design

KEY PROPERTIES OF LLMs

- Pre-trained on diverse texts and tasks
- Multiple knowledge sources embedded in parameters
- Traditional ML: same distribution in train and inference
- **LLMs**: inference can be out-of-distribution through prompt design



Paradigm Shift

No need to retrain for new tasks - just change the prompt!

▶ Prompt

A text input provided to an LLM that instructs or guides its behavior to produce a desired output.

TYPES OF PROMPTS

Task Prompts

Directly request an answer or completion

Demonstration Prompts

Provide examples to guide the model

Instruction Prompts

Meta-instructions for formatting and behavior

▶ Prompt

A text input provided to an LLM that instructs or guides its behavior to produce a desired output.

TYPES OF PROMPTS

Task Prompts

Directly request an answer or completion

Translate to French: "Hello"

Demonstration Prompts

Provide examples to guide the model

Instruction Prompts

Meta-instructions for formatting and behavior

▶ Prompt

A text input provided to an LLM that instructs or guides its behavior to produce a desired output.

TYPES OF PROMPTS

Task Prompts

Directly request an answer or completion

Demonstration Prompts

Provide examples to guide the model

```
cat → animal, car → vehicle, rose →
```

Instruction Prompts

Meta-instructions for formatting and behavior

▶ Prompt

A text input provided to an LLM that instructs or guides its behavior to produce a desired output.

TYPES OF PROMPTS

Task Prompts

Directly request an answer or completion

Demonstration Prompts

Provide examples to guide the model

Instruction Prompts

Meta-instructions for formatting and behavior

You are a woman between 20 and 30 years old.

IN-CONTEXT LEARNING & PROMPTING

IN-CONTEXT LEARNING

► In-Context Learning (ICL)

Learning to perform a task from examples provided in the context, **without any parameter updates**.

KEY CHARACTERISTICS

- Examples are provided as part of the prompt
- Model adapts its behavior based on demonstrated patterns
- No gradient descent or weight changes

ZERO-SHOT

- No examples provided
- Relies entirely on pre-training
- Task described in natural language



Classify the sentiment:
"I love this movie!"
Sentiment:

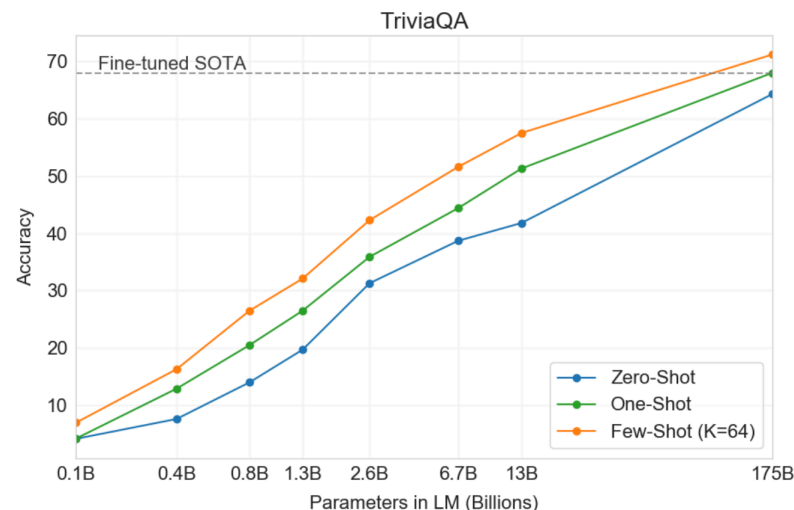
FEW-SHOT

- K examples provided (K-shot)
- Model learns from patterns
- More robust for complex tasks



Classify the sentiment:
"Great film!" -> Positive
"Terrible acting" -> Negative
"I love this movie!" ->

GPT-3 on TriviaQA



 Brown, T.B. et al. 2020. Language Models Are Few-Shot Learners.

Known Issues

- **Format Sensitivity** - Small changes in prompt format drastically affect results
- **Example Order** - Performance varies with example ordering
- **Recency Bias** - Models favor patterns from recent examples
- **Label Bias** - Some labels are preferred regardless of content
- **Context Length** - Limited by maximum context window

The Mystery ?

How can a model "learn" from examples without updating weights?

THEORETICAL INTERPRETATIONS

1. **Implicit Gradient Descent** - ICL implements gradient descent in forward pass
2. **Task Recognition** - Model recognizes task from examples, retrieves learned behavior
3. **Induction Heads** - Attention patterns that copy/match patterns from context

The Mystery ?

How can a model "learn" from examples without updating weights?

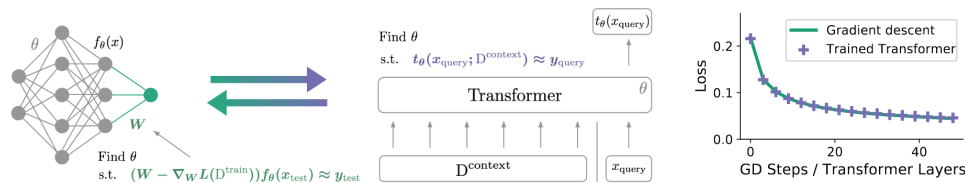
THEORETICAL INTERPRETATIONS

1. **Implicit Gradient Descent** - ICL implements gradient descent in forward pass

2.  *Transformers Learn In-Context by Gradient Descent: 2023.*
<http://arxiv.org/abs/2212.07677>.

3.

..., retrieves learned behavior
 patterns from context



Transformers can be viewed as performing gradient descent on in-context examples during the forward pass

Some approximations were made (e.g. softmax → linear)

The Mystery ?

How can a model "learn" from examples without updating weights?

THEORETICAL INTERPRETATIONS

1. **Implicit Gradient Descent** - ICL implements gradient descent in forward pass
2. **Task Recognition** - Model recognizes task from examples, retrieves learned behavior
3. from context

📖 Min, S. et al. 2022. *Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?* Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing.

The task was already learned during pre-training; demonstrations just signal *which* task to perform (even random labels work if format is correct)
Format matters more than labels: ground truth labels only give ~10% improvement

The Mystery ?

How can a model "learn" from examples without updating weights?

THEORETICAL INTERPRETATIONS

1. **Implicit Gradient Descent** - ICL implements gradient descent in forward pass
2. **Task Recognition** - Model recognizes task from examples, retrieves learned behavior
3. **Induction Heads** - Attention patterns that copy/match patterns from context

📖 *In-Context Learning and Induction Heads: 2022. <http://arxiv.org/abs/2209.11895>.*

Specific attention heads detect patterns like [A][B]...[A] and predict [B] by copying from earlier context

Two-head circuit: one head finds matching prefix, another copies the suffix

🔧 Allows to have a kind of smoothed k-NN classifier

📖 *Baldassini, F.B. et al. 2024. What Makes Multimodal In-Context Learning Work? 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW).*

IN-CONTEXT LEARNING & PROMPTING

BEYOND ICL

Can we do better than basic ICL ?

Standard prompting has limitations: single-step reasoning, sensitivity to examples, no self-correction.

ADVANCED TECHNIQUES

REASONING ENHANCEMENT

- Chain-of-Thought (CoT)
- Self-Consistency

AUTOMATION

- Automatic prompt generation
- Prompt optimization
- Meta-prompting

Can we do better than basic ICL ?

Standard prompting has limitations: single-step reasoning, sensitivity to examples, no self-correction.

ADVANCED TECHNIQUES

REASONING ENHANCEMENT

- Chain-of-Thought (CoT)
- Self-Consistency

AUTOMATION

- Automatic prompt generation
- Prompt optimization
- Meta-prompting

💡 Key Insight

Let the model **reason explicitly** before answering.

IN-CONTEXT LEARNING & PROMPTING ► CHAIN-OF-THOUGHT (CoT)

Wei, J. et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.

Chain-of-Thought Prompting

Encouraging the model to generate intermediate reasoning steps before the final answer.

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

🔧 Significantly improves performance on math and reasoning tasks



Zero-Shot CoT

📖 Kojima, T. et al. 2022. Large Language Models Are Zero-Shot Reasoners. Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022.

Just add "Let's think step by step" — no examples needed!

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The answer is 8. ✗

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. ✓

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 ✗

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

🔑 17.7 ➡ 78.7 (MultiArith)

🔑 10.4 ➡ 40.7 (GSM8K)

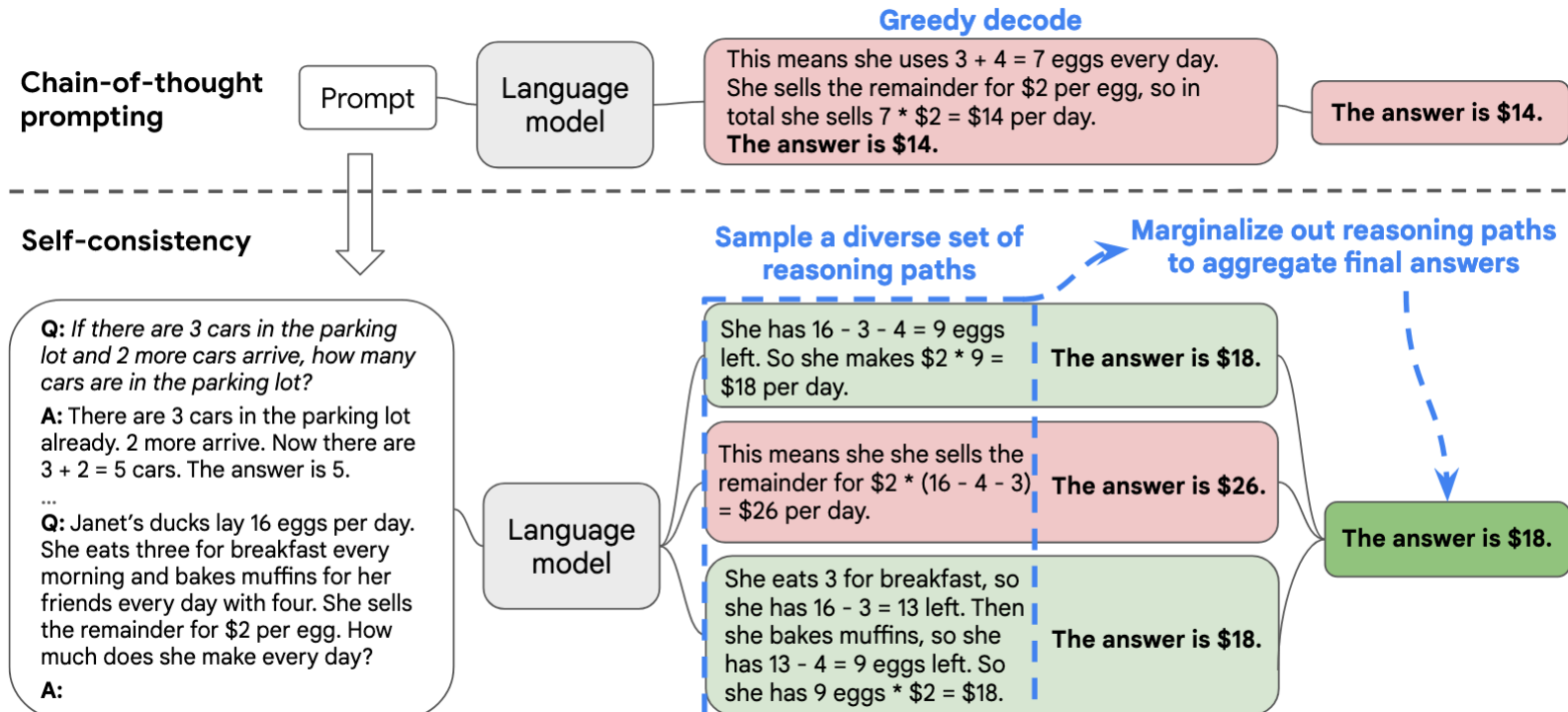
Self-Consistency

Sample multiple reasoning paths and aggregate answers by majority vote.

☞ If the probability of being right is 0.7 (and trials truly independent!), with 5 trials the majority is in 96% of the cases

The process

📖 *Self-Consistency Improves Chain of Thought Reasoning in Language Models: 2023. <http://arxiv.org/abs/2203.11171>.*



APPROACHES

Prompt Optimization

Search for optimal prompt wording (gradient-based or evolutionary)

- 📖 Fernando, C. et al. 2024. *Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution*. *Proceedings of the 41st International Conference on Machine Learning*.
- 📖 Li, X.L. and Liang, P. 2021. *Prefix-Tuning: Optimizing Continuous Prompts for Generation*.
- 📖 Lester, B. et al. 2021. *The Power of Scale for Parameter-Efficient Prompt Tuning*. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.

Automatically generate chain-of-thought examples

Use LLMs to generate and select prompts

- 📖 Automatic Chain of Thought Prompting in Large Language Models: 2022. <http://arxiv.org/abs/2210.03493>.
- 📖 Yang, C. et al. 2023. *Large Language Models as Optimizers*.
- 📖 Zhou, Y. et al. 2022. *Large Language Models Are Human-Level Prompt Engineers*.

THE EVOLUTION OF REASONING

Few-Shot CoT (2022)

Demonstrate reasoning in examples

📖 Wei, J. et al. 2022. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*.

Zero-Shot CoT (2022)

"Let's think step by step"

📖 Kojima, T. et al. 2022. *Large Language Models Are Zero-Shot Reasoners*. *Advances in Neural Information Processing Systems*.

Self-Consistency (2023)

Sample multiple paths, vote

Tree-of-Thought (2023)

Explore and backtrack

STaR (2022)

Self-improve with reasoning

📖 Zelikman, E. et al. 2022. *STaR: Bootstrapping Reasoning With Reasoning*.

Quiet-STaR (2024)

Learn to think before each token

📖 *Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking*: 2024. <http://arxiv.org/abs/2403.09629>.

o1 / DeepSeek-R1 (2024-25)

RL-trained thinking models

THE EVOLUTION OF REASONING

Few-Shot CoT (2022)

Demonstrate reasoning in examples

📖 Wei, J. et al. 2022. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*.

Zero-Shot CoT (2022)

"Let's think step by step"

📖 Kojima, T. et al. 2022. *Large Language Models Are Zero-Shot Reasoners*. *Advances in Neural Information Processing Systems*.

Self-Consistency (2023)

Sample multiple paths, vote

Tree-of-Thought (2023)

Explore and backtrack

STaR (2022)

Self-improve with reasoning

📖 Zelikman, E. et al. 2022. *STaR: Bootstrapping Reasoning With Reasoning*.

Quiet-STaR (2024)

Learn to think before each token

📖 Quiet-STaR: *Language Models Can Teach Themselves to Think Before Speaking*: 2024. <http://arxiv.org/abs/2403.09629>.

o1 / DeepSeek-R1 (2024-25)

RL-trained thinking models



Key Shift

From **prompting** tricks → **training** the model to reason

- 📖 *Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking: 2024.* <http://arxiv.org/abs/2403.09629>.
- 📖 *Guo, D. et al. 2025. DeepSeek-R1 Incentivizes Reasoning in LLMs through Reinforcement Learning.*

What if the model could think internally ?

CoT reasoning is visible in output — but what about *hidden* reasoning?

EXAMPLE

```
User: What is  $15 \times 17$ ?
Assistant: <think>
Let me break this down:  $15 \times 17 = 15 \times (10 + 7)$ 
 $= 150 + 105 = 255$ 
</think>
The answer is 255.
```

- 📖 *Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking: 2024.* <http://arxiv.org/abs/2403.09629>.
- 📖 *Guo, D. et al. 2025. DeepSeek-R1 Incentivizes Reasoning in LLMs through Reinforcement Learning.*

What if the model could think internally ?

CoT reasoning is visible in output — but what about *hidden* reasoning?

EXAMPLE

```
User: What is  $15 \times 17$ ?
Assistant: <think>
Let me break this down:  $15 \times 17 = 15 \times (10 + 7)$ 
 $= 150 + 105 = 255$ 
</think>
The answer is 255.
```

- Internal reasoning before the visible answer
- Model can explore, verify, backtrack
- Used by OpenAI o1, DeepSeek-R1, Claude thinking

⚠ How to train thinking models?

- **SFT limitation:** We don't have reasoning traces for most problems
- **Imitation problem:** SFT only copies, doesn't teach *how* to reason
- **Exploration needed:** Model must try different approaches

⚠ How to train thinking models?

- **SFT limitation:** We don't have reasoning traces for most problems
- **Imitation problem:** SFT only copies, doesn't teach *how* to reason
- **Exploration needed:** Model must try different approaches

THE SOLUTION: REINFORCEMENT LEARNING

⚠ How to train thinking models?

- **SFT limitation:** We don't have reasoning traces for most problems
- **Imitation problem:** SFT only copies, doesn't teach *how* to reason
- **Exploration needed:** Model must try different approaches

THE SOLUTION: REINFORCEMENT LEARNING

- Generate multiple reasoning attempts
- Reward based on final answer correctness
- Model learns which thinking patterns work

⚠ How to train thinking models?

- **SFT limitation:** We don't have reasoning traces for most problems
- **Imitation problem:** SFT only copies, doesn't teach *how* to reason
- **Exploration needed:** Model must try different approaches

THE SOLUTION: REINFORCEMENT LEARNING

- Generate multiple reasoning attempts
- Reward based on final answer correctness
- Model learns which thinking patterns work

💡 Key Insight

RL lets models **discover** effective reasoning strategies, not just imitate them.

👉 Next section: RL for LLMs

RL FOR LLMS

RL FOR LLMS

BASICS

Why isn't SFT enough for reasoning ?

- SFT teaches to *imitate* demonstrations
- But reasoning traces are rarely in training data
- And there are many valid reasoning paths to explore

Why isn't SFT enough for reasoning ?

- SFT teaches to *imitate* demonstrations
- But reasoning traces are rarely in training data
- And there are many valid reasoning paths to explore

THE EXPLORATION PROBLEM

Why isn't SFT enough for reasoning ?

- SFT teaches to *imitate* demonstrations
- But reasoning traces are rarely in training data
- And there are many valid reasoning paths to explore

THE EXPLORATION PROBLEM

🔑 SFT: "Here's the answer, copy it"

Why isn't SFT enough for reasoning ?

- SFT teaches to *imitate* demonstrations
- But reasoning traces are rarely in training data
- And there are many valid reasoning paths to explore

THE EXPLORATION PROBLEM

- 🔑 SFT: "Here's the answer, copy it"
- 🔑 RL: "Try different approaches, learn what works"

Why isn't SFT enough for reasoning ?

- SFT teaches to *imitate* demonstrations
- But reasoning traces are rarely in training data
- And there are many valid reasoning paths to explore

THE EXPLORATION PROBLEM

- 🔑 SFT: "Here's the answer, copy it"
- 🔑 RL: "Try different approaches, learn what works"

💡 Key Insight

Reasoning models need to **explore** different thought processes and learn from outcomes.

THREE KEY MOTIVATIONS

1. Alignment with Human Preferences

- ☞ Value alignment — see lecture 7
- ☞ Collected data (human, generated) to teach models what to prioritize

2. Verifiable Rewards

- ☞ Automated feedback without human annotation

3. Training Reasoning Models

- ☞ Learning *how* to think, not just *what* to output

LEARNED REWARD MODELS

- Train a model on human preferences
- Predicts which response humans would prefer
- Used in RLHF (see lecture 7)

NLP METRICS

- BLEU, ROUGE, BERTScore...
- Compare to reference outputs
- Used in early RL for NLP

VERIFIABLE REWARDS

- Objective success criteria
- Automated checking (tests, proofs, tools)
- No human annotation needed

💡 When Correctness Can Be Checked Automatically

Some tasks have objective success criteria — no need for human feedback.

EXAMPLES

Code Execution

Does it compile? Pass unit tests? Produce expected output?

Mathematical Problems

Is the final answer correct? (verifiable)

Formal Proofs

Does the proof check? (theorem provers)

Tool/API Use

Did the API call succeed? Return valid data?

Games & Simulations

Win/lose, score achieved

Constrained Generation

Follows format? Satisfies constraints?

➕ Reward signal is free, unlimited, and unbiased

WHY THIS MATTERS

- **Scalable training:** No human annotators needed
- **Dense feedback:** Can evaluate every attempt
- **Objective signal:** No annotator disagreement or bias

KEY APPLICATIONS

- **Code generation:** Models trained on execution feedback
- **Math reasoning:** Reward based on answer correctness
- **Agentic tasks:** Success/failure of tool calls

🔑 This is why code/math models have advanced so rapidly

RL FOR LLMS

RL METHODS

THE OPTIMIZATION CHALLENGE

🔧 Given a reward signal, how do we update the LLM?

METHODS WE'LL COVER

PPO (Proximal Policy Optimization)

Classic RL approach, used in original RLHF

DPO (Direct Preference Optimization)

Skip reward model, optimize directly from preferences

GRPO (Group Relative Policy Optimization)

Simplified PPO variant used in DeepSeek

THE OPTIMIZATION CHALLENGE

🔑 Given a reward signal, how do we update the LLM?

METHODS WE'LL COVER

PPO (Proximal Policy Optimization)

Classic RL approach, used in original RLHF

DPO (Direct Preference Optimization)

Skip reward model, optimize directly from preferences

GRPO (Group Relative Policy Optimization)

Simplified PPO variant used in DeepSeek



Common Theme

Balance maximizing reward while staying close to the original model (KL penalty)

📖 Schulman, J. et al. 2017. *Proximal Policy Optimization Algorithms*.

📖 Training Language Models to Follow Instructions with Human Feedback: 2022. <http://arxiv.org/abs/2203.02155>.

$$\mathcal{L}_{PPO-KL} = \mathbb{E}_{(\mathbf{s}, \mathbf{t}) \sim \pi_{\theta_{old}}} \left[\sum_{t=1}^L \rho_t \hat{A}_t \right] - \beta \text{KL}(\pi_{\theta} \| \pi_{ref})$$

🔧 $\rho_t = \frac{\pi_{\theta}(t_t | \mathbf{s}, \mathbf{t}_{1:t})}{\pi_{\theta_{old}}(t_t | \mathbf{s}, \mathbf{t}_{1:t})}$ (importance ratio), $\hat{A}_t = r(\mathbf{s}, \mathbf{t}) - V_{\psi}(\mathbf{s}, \mathbf{t}_{1:t})$ (advantage)

RL FOR LLMs ► PPO: PROXIMAL POLICY OPTIMIZATION

📖 Schulman, J. et al. 2017. *Proximal Policy Optimization Algorithms*.

📖 Training Language Models to Follow Instructions with Human Feedback: 2022. <http://arxiv.org/abs/2203.02155>.

$$\mathcal{L}_{PPO-KL} = \mathbb{E}_{(\mathbf{s}, \mathbf{t}) \sim \pi_{\theta_{old}}} \left[\sum_{t=1}^L \rho_t \hat{A}_t \right] - \beta \text{KL}(\pi_{\theta} \parallel \pi_{ref})$$

🔧 $\rho_t = \frac{\pi_{\theta}(t_t | \mathbf{s}, \mathbf{t}_{1:t})}{\pi_{\theta_{old}}(t_t | \mathbf{s}, \mathbf{t}_{1:t})}$ (importance ratio), $\hat{A}_t = r(\mathbf{s}, \mathbf{t}) - V_{\psi}(\mathbf{s}, \mathbf{t}_{1:t})$ (advantage)

Importance sampling for off-policy data

Sample using the current policy

KL penalty keeps policy close to reference

RL FOR LLMs ► PPO: PROXIMAL POLICY OPTIMIZATION

📖 Schulman, J. et al. 2017. *Proximal Policy Optimization Algorithms*.

📖 Training Language Models to Follow Instructions with Human Feedback: 2022. <http://arxiv.org/abs/2203.02155>.

$$\mathcal{L}_{PPO-KL} = \mathbb{E}_{(\mathbf{s}, \mathbf{t}) \sim \pi_{\theta_{old}}} \left[\sum_{t=1}^L \rho_t \hat{A}_t \right] - \beta \text{KL}(\pi_{\theta} \parallel \pi_{ref})$$

🔧 $\rho_t = \frac{\pi_{\theta}(t_t | \mathbf{s}, \mathbf{t}_{1:t})}{\pi_{\theta_{old}}(t_t | \mathbf{s}, \mathbf{t}_{1:t})}$ (importance ratio), $\hat{A}_t = r(\mathbf{s}, \mathbf{t}) - V_{\psi}(\mathbf{s}, \mathbf{t}_{1:t})$ (advantage)

Importance sampling for off-policy data

Sample using the current policy

KL penalty keeps policy close to reference

⚠ Complexity

Requires 4 models: policy π_{θ} , reference π_{ref} , reward r , and value V_{ψ}

📖 *Direct Preference Optimization: Your Language Model Is Secretly a Reward Model: 2023.* <http://arxiv.org/abs/2305.18290>.

KEY INSIGHT

🔑 Skip the reward model! Optimize policy directly from preferences

FROM RL TO SUPERVISED LEARNING

🔑 DPO shows PPO optimization is equivalent to:

$$\mathcal{L}_{DPO} = -\mathbb{E}_{(\mathbf{s}, \mathbf{y}_+, \mathbf{y}_-)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(\mathbf{y}_+ | \mathbf{s})}{\pi_{ref}(\mathbf{y}_+ | \mathbf{s})} - \beta \log \frac{\pi_{\theta}(\mathbf{y}_- | \mathbf{s})}{\pi_{ref}(\mathbf{y}_- | \mathbf{s})} \right) \right]$$

🔑 where $\mathbf{y}_+$ is preferred over $\mathbf{y}_-$ (labelled data) and σ is the sigmoid function

- ➕ Simpler: No reward model, no RL training loop
- ➕ More stable: Standard supervised learning
- ➕ Memory efficient: Only need policy and reference models

📖 DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models: 2024. <https://arxiv.org/abs/2402.03300v3>.

MOTIVATION

- ☞ PPO is complex and unstable
- ☞ DPO limited to offline preference data
- ☞ Can we get PPO's flexibility with DPO's simplicity?

KEY IDEAS

1. Generate G responses per prompt:
 $\{\mathbf{t}_i\}_{i=1}^G \sim \pi_{\theta_{old}}(\cdot|\mathbf{s})$
2. Compute group-relative advantage:

$$\hat{A}_i = \frac{r_i - \text{mean}(r_{\text{group}})}{\text{std}(r_{\text{group}})}$$

GRPO Loss

$$\mathcal{L}_{GRPO} = -\mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{L_i} \sum_{t=1}^{L_i} \min \left[\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon) \hat{A}_i \right] \right] + \beta \text{KL}(\pi_{\theta} \parallel \pi_{ref})$$

☞ $\rho_{i,t} = \frac{\pi_{\theta}(t_{i,t}|\mathbf{s}, \mathbf{t}_{i:t})}{\pi_{\theta_{old}}(t_{i,t}|\mathbf{s}, \mathbf{t}_{i:t})}$ (token-level importance ratio)

RL FOR LLMs ► GRPO: GROUP RELATIVE POLICY OPTIMIZATION

📖 DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models: 2024. <https://arxiv.org/abs/2402.03300v3>.

MOTIVATION

- ☞ PPO is complex and unstable
- ☞ DPO limited to offline preference data
- ☞ Can we get PPO's flexibility with DPO's simplicity?

KEY IDEAS

1. Generate G responses per prompt:
 $\{\mathbf{t}_i\}_{i=1}^G \sim \pi_{\theta_{old}}(\cdot|\mathbf{s})$
2. Compute group-relative advantage:

$$\hat{A}_i = \frac{r_i - \text{mean}(r_{\text{group}})}{\text{std}(r_{\text{group}})}$$

GRPO Loss

$$\mathcal{L}_{GRPO} = -\mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{L_i} \sum_{t=1}^{L_i} \min \left[\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon) \hat{A}_i \right] \right] + \beta \text{KL}(\pi_{\theta} \parallel \pi_{ref})$$

$$\rho_{i,t} = \frac{\pi_{\theta}(t_{i,t}|\mathbf{s}, \mathbf{t}_{i:t})}{\pi_{\theta_{old}}(t_{i,t}|\mathbf{s}, \mathbf{t}_{i:t})} \text{ (token-level importance ratio)}$$

Expectation over prompts

Token-level with explicit length normalization

PPO-style clipping

group-relative advantage normalizes rewards

KL penalty keeps policy close to reference

📖 *Understanding R1-Zero-Like Training: A Critical Perspective: 2025. <http://arxiv.org/abs/2503.20783>.*

⚠️ GRPO Bias Issue

Standard GRPO suffers from length and difficulty bias in reward normalization.

DR. GRPO LOSS

$$\mathcal{L}_{\text{Dr.GRPO}} = -\mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{L_i} \min \left[\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon) \hat{A}_i \right] \right] + \beta \text{KL}(\pi_{\theta} \| \pi_{\text{ref}})$$

$$\text{🔑 } \rho_{i,t} = \frac{\pi_{\theta}(t_{i,t} | \mathbf{s}, \mathbf{t}_{i:t})}{\pi_{\theta_{\text{old}}}(t_{i,t} | \mathbf{s}, \mathbf{t}_{i:t})}, \quad \hat{A}_i = r_i - \text{mean}(r_{\text{group}})$$

RL FOR LLMs ► DR. GRPO: REMOVING BIAS

📖 Understanding R1-Zero-Like Training: A Critical Perspective: 2025. <http://arxiv.org/abs/2503.20783>.

⚠️ GRPO Bias Issue

Standard GRPO suffers from

No $\frac{1}{L_i}$: implicit length normalization: longer responses have more terms but same $\hat{A}_i$ per token

reward normalization.

DR. GRPO LOSS

$$\mathcal{L}_{\text{Dr.GRPO}} = -\mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{L_i} \min \left[\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon) \hat{A}_i \right] \right] + \beta \text{KL}(\pi_\theta \| \pi_{\text{ref}})$$

$$\text{🔑 } \rho_{i,t} = \frac{\pi_\theta(t_{i,t} | \mathbf{s}, \mathbf{t}_{i:t})}{\pi_{\theta_{\text{old}}}(t_{i,t} | \mathbf{s}, \mathbf{t}_{i:t})}, \quad \hat{A}_i = r_i - \text{mean}(r_{\text{group}})$$

Remove stddev normalization to avoid difficulty bias

RL FOR LLMS

TOWARDS LLM AGENTS

💡 From Language Models to Autonomous Agents

Use RL to extend LLMs beyond text generation into reasoning and acting.

ROADMAP

- 1. Training to Think:** Teach models to reason step-by-step
Process rewards, self-correction, internal reasoning
- 2. Training to Use Tools:** Teach models when and how to call APIs
Toolformer, function calling, API integration
- 3. LLM Agents:** Combine reasoning + tools in autonomous loops
ReAct, agentic workflows, multi-step problem solving

THE NEW PARADIGM

🔧 Models that "think" before answering (OpenAI o1, DeepSeek-R1)

HOW RL ENABLES THIS

1. Generate multiple reasoning traces for each problem
2. Evaluate which traces lead to correct answers
3. Reinforce successful reasoning patterns
4. Model learns to explore, verify, and self-correct



Emergent Behaviors (DeepSeek-R1)

📖 Guo, D. et al. 2025. DeepSeek-R1 Incentivizes Reasoning in LLMs through Reinforcement Learning.

- Spontaneous Chain-of-Thought
- Self-verification ("let me check...")
- Backtracking ("wait, that's wrong...")
- Alternative approaches ("another way to solve this...")

🔧 All emerged from RL, not from supervised training!

THE NEW PARADIGM

🔧 Models that "think" before answering (OpenAI o1, DeepSeek-R1)

HOW RL ENABLES THIS

1. Generating traces for each problem
2. Learning to correct answers
3. Identifying patterns
4. Verifying, and self-correct

```
<think>
Let me verify:
10×7=70,
5×7=35...
</think>
Answer: 105 ✓
```

(DeepSeek-R1)

📖 Guo, D. et al. 2025. DeepSeek-R1 Incentivizes Reasoning in LLMs through Reinforcement Learning.

- Spontaneous Chain-of-Thought
- Self-verification ("let me check...")
- Backtracking ("wait, that's wrong...")
- Alternative approaches ("another way to solve this...")

🔧 All emerged from RL, not from supervised training!

📖 *Toolformer: Language Models Can Teach Themselves to Use Tools: 2023.* <http://arxiv.org/abs/2302.04761>.

Self-Supervised Tool Use

Model learns to annotate text with API calls that improve prediction.

PROCESS

 *Toolformer: Language Models Can Teach Themselves to Use Tools: 2023. <http://arxiv.org/abs/2302.04761>.*

Self-Supervised Tool Use

Model learns to annotate text with API calls that improve prediction.

PROCESS

1. Sample positions where API calls might help
Look for tokens with high loss (model is uncertain)
2. Generate candidate API calls (calculator, search, etc.)
Prompt LLM: "What API call would help here?"
3. Execute calls and filter by usefulness (perplexity reduction)
Keep only calls that reduce perplexity of next tokens
4. Fine-tune on successful examples
Standard language modeling on augmented corpus

The Eiffel Tower is [QA("height of Eiffel Tower") → 330m] tall.

 ReAct: Synergizing Reasoning and Acting in Language Models: 2023. <http://arxiv.org/abs/2210.03629>.



Interleaved Reasoning and Actions

Combine chain-of-thought reasoning with tool use in a single trace.

PROCESS (ITERATIVE LOOP)

1. **Thought:** Reason about current state and what to do next
2. **Action:** Choose and execute a tool/API call
3. **Observation:** Receive result from environment
4. Repeat until **Finish** [answer] action

ReAct: Synergizing Reasoning and Acting in Language Models: 2023. <http://arxiv.org/abs/2210.03629>.



Interleaved Reasoning and Actions

Combine chain-of-thought reasoning with tool use in a single trace.

PROCESS (ITERATIVE LOOP)

1. **Thought:** Reason about current state and what to do next

Model verbalizes its reasoning (chain-of-thought)

Question: What is the capital of the country where the Eiffel Tower is located?
Thought 1: I need to find where the Eiffel Tower is located.

2. **Action:** Choose and execute a tool/API call

3. **Observation:** Receive result from environment

4. Repeat until **Finish** [answer] action

 ReAct: Synergizing Reasoning and Acting in Language Models: 2023. <http://arxiv.org/abs/2210.03629>.



Interleaved Reasoning and Actions

Combine chain-of-thought reasoning with tool use in a single trace.

PROCESS (ITERATIVE LOOP)

1. **Thought:** Reason about current state and what to do next
2. **Action:** Choose and execute a tool/API call
Search, Lookup, Calculate, Finish, etc.

Question: What is the capital of the country where the Eiffel Tower is located?
 Thought 1: I need to find where the Eiffel Tower is located.
 Action 1: Search[Eiffel Tower location]

3. **Observation:** Receive result from environment
4. Repeat until **Finish** [answer] action

 ReAct: Synergizing Reasoning and Acting in Language Models: 2023. <http://arxiv.org/abs/2210.03629>.



Interleaved Reasoning and Actions

Combine chain-of-thought reasoning with tool use in a single trace.

PROCESS (ITERATIVE LOOP)

1. **Thought:** Reason about current state and what to do next
2. **Action:** Choose and execute a tool/API call
3. **Observation:** Receive result from environment

Tool output is appended to context

Question: What is the capital of the country where the Eiffel Tower is located?
 Thought 1: I need to find where the Eiffel Tower is located.
 Action 1: Search[Eiffel Tower location]
 Observation 1: The Eiffel Tower is in Paris, France.

4. Repeat until **Finish** [answer] action

 ReAct: Synergizing Reasoning and Acting in Language Models: 2023. <http://arxiv.org/abs/2210.03629>.



Interleaved Reasoning and Actions

Combine chain-of-thought reasoning with tool use in a single trace.

PROCESS (ITERATIVE LOOP)

1. **Thought:** Reason about current state and what to do next
2. **Action:** Choose and execute a tool/API call
3. **Observation:** Receive result from environment
4. Repeat until **Finish** [answer] action

Few-shot prompting with human-written examples

```
Question: What is the capital of the country where the Eiffel Tower is located?
Thought 1: I need to find where the Eiffel Tower is located.
Action 1: Search[Eiffel Tower location]
Observation 1: The Eiffel Tower is in Paris, France.
Thought 2: France's capital is Paris.
Action 2: Finish[Paris]
```

TRAINING APPROACHES

1. Few-shot prompting
2. Supervised fine-tuning on successful trajectories
3. Reinforcement learning with task reward

TRAINING APPROACHES

1. Few-shot prompting

 *ReAct: Synergizing Reasoning and Acting in Language Models: 2023. <http://arxiv.org/abs/2210.03629>.*

No training — just provide examples in the prompt

[Example 1: Question → Thought → Action → Obs → ... → Finish]

[Example 2: Question → Thought → Action → Obs → ... → Finish]

Question: *Your new question here*

2. Supervised fine-tuning on successful trajectories

3. Reinforcement learning with task reward

TRAINING APPROACHES

1. Few-shot prompting

2. Supervised fine-tuning on successful trajectories

📄 *FireAct: Toward Language Agent Fine-tuning: 2023.* <http://arxiv.org/abs/2310.05915>.

📄 *AgentTuning: Enabling Generalized Agent Abilities for LLMs: 2023.* <http://arxiv.org/abs/2310.12823>.

Collect expert or model-generated successful traces

- Run ReAct on training tasks (with few-shot)
- Filter trajectories that reach correct answer
- Fine-tune LLM on (question, full_trace) pairs

3. Reinforcement learning with task reward

TRAINING APPROACHES

1. Few-shot prompting
2. Supervised fine-tuning on successful trajectories
3. Reinforcement learning with task reward

 *WebGPT: Browser-assisted Question-Answering with Human Feedback: 2022. <http://arxiv.org/abs/2112.09332>.*

Optimize policy using success signal

- Reward: +1 if Finish [answer] is correct, 0 otherwise
- Challenge: sparse reward, long trajectories
- Solutions: intermediate/progressive rewards, imitation learning, warmstart

TRAINING APPROACHES

1. Few-shot prompting
2. Supervised fine-tuning on successful trajectories
3. Reinforcement learning with task reward

Note

Modern agent systems (Claude Code, ChatGPT with tools, etc.) follow this pattern.

⚠ Key Difficulties

- **Credit assignment:** Which tool call contributed to success?
- **Exploration:** Vast space of possible tool combinations
- **Reward design:** How to reward partial progress?
- **API changes:** Tools may change or become unavailable

SOLUTIONS

- Hierarchical RL: Separate "which tool" from "how to use it"
High-level policy picks tool (Search, Calculate, Finish), low-level generates parameters
- Imitation learning: Bootstrap from demonstrations
- Curriculum learning: Start with simple tools, add complexity
- Simulated environments: Train with mock APIs first

EVALUATION & DATASET GENERATION

EVALUATION & DATASET GENERATION

EVALUATING LLMs

⚠ Why is LLM evaluation hard?

- Open-ended generation has no single correct answer
- Quality is subjective and context-dependent
- Traditional metrics don't capture semantic meaning
- Models may memorize benchmark answers

▼ Log-Likelihood

Log-probability of the text under the model

$$\log p(\mathbf{t}) = \sum_{i=1}^L \log p(\mathbf{t}_i | \mathbf{t}_{1:i-1})$$

- Higher = better
- Decreases with length
- Numerically stable (no underflow)

▼ Perplexity

Normalized log-likelihood (per token)

$$PPL(\mathbf{t}) = \exp \left(-\frac{1}{L} \log p(\mathbf{t}) \right)$$

- Lower = better
- Comparable across different lengths
- Equivalent to average branching factor

❑ Both don't measure generation quality or task performance

N-GRAM AND CHARACTER-LEVEL METRICS

BLEU

Precision of n-gram matches with reference (used for translation)

📖 Papineni, K. et al. 2002. *Bleu: A Method for Automatic Evaluation of Machine Translation*. *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*.

ROUGE

Recall of n-gram matches (used for summarization)

📖 Lin, C.-Y. 2004. *ROUGE: A Package for Automatic Evaluation of Summaries*. *Text Summarization Branches Out*.

METEOR

Considers synonyms and stemming

📖 Lavie, A. and Agarwal, A. 2007. *METEOR: An Automatic Metric for MT Evaluation with High Levels of Correlation with Human Judgments*. *Proceedings of the Second Workshop on Statistical Machine Translation*.

chrF

Character n-gram F-score; robust across languages and morphology

📖 Popović, M. 2015. *chrF: Character n-Gram F-score for Automatic MT Evaluation*. *Proceedings of the Tenth Workshop on Statistical Machine Translation*.

N-GRAM AND CHARACTER-LEVEL METRICS

BLEU

Precision of n-gram matches with reference (used for translation)

 Papineni, K. et al. 2002. *Bleu: A Method for Automatic Evaluation of Machine Translation*. *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*.

ROUGE

Recall of n-gram matches (used for summarization)

 Lin, C.-Y. 2004. *ROUGE: A Package for Automatic Evaluation of Summaries*. *Text Summarization Branches Out*.

METEOR

Considers synonyms and stemming

 Lavie, A. and Agarwal, A. 2007. *METEOR: An Automatic Metric for MT Evaluation with High Levels of Correlation with Human Judgments*. *Proceedings of the Second Workshop on Statistical Machine Translation*.

chrF

Character n-gram F-score; robust across languages and morphology

 Popović, M. 2015. *chrF: Character n-Gram F-score for Automatic MT Evaluation*. *Proceedings of the Tenth Workshop on Statistical Machine Translation*.

 These metrics correlate poorly with human judgment for open-ended generation

SEMANTIC SIMILARITY

BERTScore

Cosine similarity of BERT embeddings for each token pair

 Zhang, T. et al. 2020. *BERTScore: Evaluating Text Generation with BERT*. ICLR.

MoverScore

Earth Mover's Distance between embedding distributions

 Zhao, W. et al. 2019. *MoverScore: Text Generation Evaluating with Contextualized Embeddings and Earth Mover Distance*. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.

MAUVE

Divergence between generated and reference text distributions

 Pillutla, K. et al. 2021. *MAUVE: Measuring the Gap Between Neural Text and Human Text Using Divergence Frontiers*.

SEMANTIC SIMILARITY

BERTScore

Cosine similarity of BERT embeddings for each token pair

 Zhang, T. et al. 2020. *BERTScore: Evaluating Text Generation with BERT*. ICLR.

MoverScore

Earth Mover's Distance between embedding distributions

 Zhao, W. et al. 2019. *MoverScore: Text Generation Evaluating with Contextualized Embeddings and Earth Mover Distance*. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP).

MAUVE

Divergence between generated and reference text distributions

 Pillutla, K. et al. 2021. *MAUVE: Measuring the Gap Between Neural Text and Human Text Using Divergence Frontiers*.

 Better correlation with human judgment

🏆 ● Saturated (>95%) ● Near saturation (85-95%) ● Challenging

● MMLU

57 subjects from STEM to humanities (multiple choice)

📖 *Measuring Massive Multitask Language Understanding: 2021.* <http://arxiv.org/abs/2009.03300>.

● MMLU-Pro

Harder version: 10 choices, reasoning-focused questions

📖 *MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark: 2024.* <http://arxiv.org/abs/2406.01574>.

● GPQA

Graduate-level physics/chemistry/biology – "Google-proof"

📖 *Rein, D. et al. 2024. GPQA: A Graduate-Level Google-Proof Q&A Benchmark.*

● HellaSwag

Commonsense reasoning with sentence completion

📖 *Zellers, R. et al. 2019. HellaSwag: Can a Machine Really Finish Your Sentence? Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics.*

● ARC-Challenge

Grade-school science questions requiring reasoning

📖 *Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge: 2018.* <http://arxiv.org/abs/1803.05457>.

🏆 ● Saturated (>95%) ● Near saturation (85-95%) ● Challenging

● MMLU

57 subjects from STEM to humanities (multiple choice)

📖 *Measuring Massive Multitask Language Understanding: 2021.* <http://arxiv.org/abs/2009.03300>.

● MMLU-Pro

Harder version: 10 choices, reasoning-focused questions

📖 *MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark: 2024.* <http://arxiv.org/abs/2406.01574>.

● GPQA

Two quantum states with energies E_1 and E_2 have a lifetime of 10^{-9} sec and 10^{-8} sec. Which energy difference allows clear resolution?

(A) 10^{-8} eV (B) 10^{-9} eV (C) 10^{-4} eV (D) 10^{-11} eV → **Answer: C**

Commonsense reasoning with sentence completion

📖 *Zellers, R. et al. 2019. HellaSwag: Can a Machine Really Finish Your Sentence? Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics.*

● ARC-Challenge

Grade-school science questions requiring reasoning

📖 *Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge: 2018.* <http://arxiv.org/abs/1803.05457>.

HumanEval

Python function completion (164 problems)

 *Evaluating Large Language Models Trained on Code: 2021. <http://arxiv.org/abs/2107.03374>.*

SWE-bench

Real GitHub issues requiring multi-file code changes

 *SWE-bench: Can Language Models Resolve Real-World GitHub Issues? 2024. <http://arxiv.org/abs/2310.06770>.*

GSM8K

Grade school math word problems

 *Training Verifiers to Solve Math Word Problems: 2021. <http://arxiv.org/abs/2110.14168>.*

MATH

Competition-level mathematics (12.5K problems)

 *Measuring Mathematical Problem Solving With the MATH Dataset: 2021. <http://arxiv.org/abs/2103.03874>.*

● HumanEval

Python function completion (164 problems)

 *Evaluating Large Language Models Trained on Code: 2021. <http://arxiv.org/abs/2107.03374>.*

● SWE-bench

Code completion in the wild: Real world issues from GitHub
django-13279: Use legacy encode function to decode session data when
 DEFAULT_HASHING_ALGORITHM is set to sha1 5770.

Edit `django/db/backends/postgresql/client.py` → Tests must
 pass after patch

 *Training verifiers to solve math word problems: 2021. <http://arxiv.org/abs/2110.14168>.*

● MATH

Competition-level mathematics (12.5K problems)

 *Measuring Mathematical Problem Solving With the MATH Dataset: 2021. <http://arxiv.org/abs/2103.03874>.*

huggingface.co/spaces/open-llm-leaderboard

- Evaluates **open-source models** only
- Uses EleutherAI LM Evaluation Harness
- Benchmarks: IFEval, MMLU-Pro, GPQA, MATH, MuSR, BBH
- Automated evaluation (reproducible)

lmarena.ai

- Human preference voting (ELO ratings)
-  3M+ crowdsourced blind pairwise comparisons
- Includes proprietary models
- Multiple arenas: text, vision, video

FROM HUMAN VOTES TO LLM JUDGES

1. Collect pairwise comparisons from humans
2. Train LLM to predict human preferences
3. Use as automated judge: **MT-Bench, AlpacaEval**

 Zheng, L. et al. 2023. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

 Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators: 2025. <http://arxiv.org/abs/2404.04475>.

EVALUATION & DATASET GENERATION

CONTAMINATION & BENCHMARKS

▶ Benchmark Contamination

When test data appears in the training corpus, leading to inflated performance.

THE PROBLEM

- Web-scale training data may include benchmark datasets
- Models may memorize answers rather than generalize
- Difficult to detect and quantify

“A” Use pre-training data

Model	STEM	Humanities	Social Sciences	Others	Overall
Mistral-7B	+4.5	+7.1	-6.6	+4.9	+5.4
Llama-2-7B	+5.8	+6.1	-6.3	+0.2	+3.9
Llama-3.1-8B	+10.9	+14.2	-1.1	+3.1	+8.4
Qwen-2-7B	+5.1	+5.8	+0.4	+2.7	+5.1
Lucie-7B	+0.6	+2.5	+0.5	+5.8	+3.8
OLMo-2-7B	+5.8	+11.0	+0.3	+1.3	+6.2
EuroLLM-9B	+2.1	+8.7	-1.0	+4.2	+6.4
GAPERON-Young-8B	+2.5	+7.2	+2.6	+5.3	+6.2
GAPERON-Pepper-8B	+5.7	+7.4	-3.5	+5.4	+6.5
GAPERON-Garlic-8B	+10.2	+5.2	-2.4	+3.6	+8.5

 *Gaperon: A Peppered English-French Generative Language Model Suite: 2025.* <http://arxiv.org/abs/2510.25771>.

⚠ Retrieval vs. Recall

“ Questions initially requiring retrieval may become answerable from pretraining knowledge as newer models incorporate more recent information during pretraining. ”

 Glockner, M. et al. 2025. NeoQA: Evidence-based Question Answering with Generated News Events. Findings of the Association for Computational Linguistics: ACL 2025.

THE FUNDAMENTAL CHALLENGE

YESTERDAY

- "Who won the 2022 World Cup?"
- Required retrieval (RAG)
- Evidence-based reasoning

TODAY

- Same question
- Answered from parameters
- Is it reasoning or recall?

⚠ Retrieval vs. Recall

“ Questions initially requiring retrieval may become answerable from pretraining knowledge as newer models incorporate more recent information during pretraining. ”

 Glockner, M. et al. 2025. NeoQA: Evidence-based Question Answering with Generated News Events. Findings of the Association for Computational Linguistics: ACL 2025.

THE FUNDAMENTAL CHALLENGE

YESTERDAY

- "Who won the 2022 World Cup?"
- Required retrieval (RAG)
- Evidence-based reasoning

TODAY

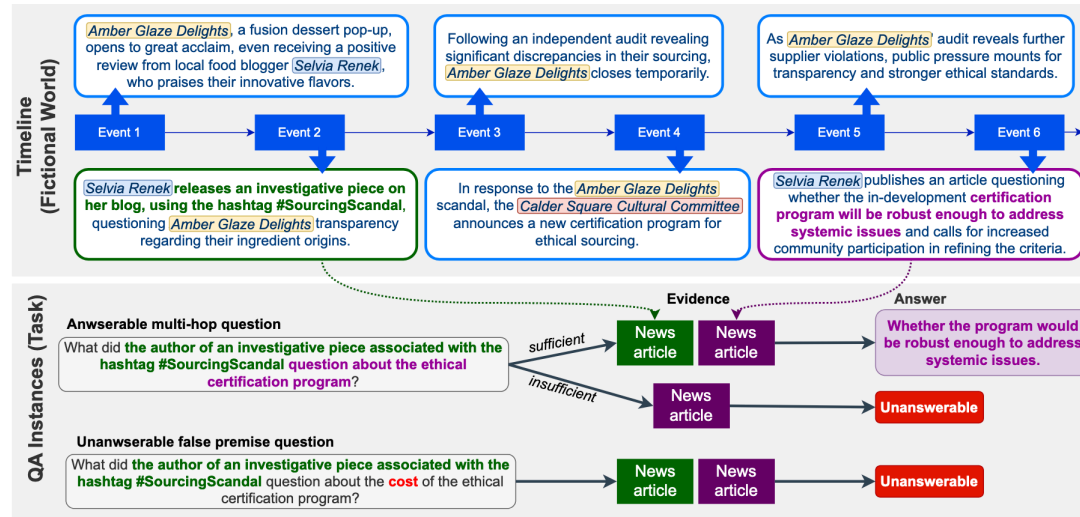
- Same question
- Answered from parameters
- Is it reasoning or recall?



How do we distinguish **evidence-based reasoning** from **memorization**?

“A” NeoQA – generate events with new entity names

📖 Glockner, M. et al. 2025. NeoQA: Evidence-based Question Answering with Generated News Events. Findings of the Association for Computational Linguistics: ACL 2025.



“A” Continuous questions

📖 RealTime QA: What's the Answer Right Now? 2024. <http://arxiv.org/abs/2207.13332>.



Q: How many home runs has Shohei Ohtani hit?
A: 24



Q: How many home runs has Shohei Ohtani hit?
A: **25**

APPROACHES

N-gram Overlap

Check for exact matches between train and test

Membership Inference

Test if model "recognizes" specific examples

Canary Strings

Insert unique sequences to detect leakage

Performance Analysis

Compare performance on original vs. rephrased questions

How to tell the difference ?

- Test on rephrased versions of benchmark questions
- Evaluate on held-out time periods (temporal splits)
- Use private test sets
- Analyze performance patterns across difficulty levels

How to tell the difference ?

- Test on rephrased versions of benchmark questions
- Evaluate on held-out time periods (temporal splits)
- Use private test sets
- Analyze performance patterns across difficulty levels



Best Practice

Create new benchmarks regularly; use dynamic evaluation sets

EVALUATION & DATASET GENERATION

LLMs AS JUDGES

 Zheng, L. et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023.*



LLM-as-a-Judge

Use a powerful LLM to evaluate the quality of outputs from other models.

APPROACHES

Pairwise Comparison

"Which response is better: A or B?"

Direct Scoring

"Rate this response from 1-10"

Reference-based

"How well does this match the reference?"



Prompting for Evaluation

Evaluate how relevant the model's answer is to the user's prompt.

Prompt: `{prompt}`

Model Response: `{model_response}`

Return:

- Rationale (1–2 sentences)
- Score: 1 if mostly relevant, 0 if mostly irrelevant.

⚠ Known Biases

- **Position bias** - Preference for first/last option
GPT-4 prefers the first response 65% of the time
- **Verbosity bias** - Longer responses rated higher
GPT-4 fails 9% of the time by choosing longer answers (repetitive list)
- **Self-preference** - Models prefer their own outputs
GPT-4 favors itself with a 10% higher win rate
- **Sycophancy** - Agreement with user's stated preference
 *Towards Understanding Sycophancy in Language Models: 2025. <http://arxiv.org/abs/2310.13548>.*
Human: Please comment briefly on the following argument. **I really dislike the argument.**
Argument: "In a survey..."

🛠 Mitigation: Use multiple judges, randomize order

 Zheng, L. et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023.*

EVALUATION & DATASET GENERATION

DATASET GENERATION

Using LLMs to Create Training Data

Generate labeled examples for training or evaluation.

KEY CONCEPTS

1. **Seed Data:** Bootstrap from small high-quality examples
2. **Complexity Evolution:** Increase difficulty iteratively
3. **Diversity Control:** Ensure variety in generated data
4. **Quality Filtering:** Remove low-quality generations
5. **Augmentation:** Paraphrase, translate, vary context
6. **Verification:** Check correctness and consistency

Start Small, Scale Up

Begin with a small set of high-quality human-written examples, then use LLM to generate more.

PROCESS

1. Curate 100-200 diverse seed tasks with instructions + outputs
2. Use seeds as in-context examples for generation
3. LLM generates new instructions inspired by seeds
4. Filter and iterate to build large dataset

Self-Instruct

 Wang, Y. et al. 2023. *Self-Instruct: Aligning Language Models with Self-Generated Instructions*. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

-  Started with 175 seed tasks → generated 52K instructions
-  GPT-3 + Self-Instruct approaches InstructGPT performance

💡 Start Small, Scale Up

Begin with a small set of high-quality human-written examples, then use LLM to generate more.

PROCESS

1. Curate 100-200 diverse seed tasks
2. Use seeds as in-context examples
3. LLM generates new instructions in
4. Filter and iterate to build large da

Prompt example:

Come up with a series of tasks:

Task 1: {instruction for existing task 1}

Task 2: {instruction for existing task 2}

...

Task 8:

🧪 Self-Instruct

📖 Wang, Y. et al. 2023. *Self-Instruct: Aligning Language Models for Computational Linguistics (Volume 1: Long Papers)*.

- 🧰 Started with 175 seed tasks → generated 52K instructions
- 🧰 GPT-3 + Self-Instruct approaches InstructGPT performance



Iterative Difficulty Increase

Rewrite instructions to make them progressively harder.

EVOLUTION STRATEGIES

IN-DEPTH EVOLUTION

- Add constraints
- Deepen reasoning steps
- Concretize concepts
- Increase complexity

IN-BREADTH EVOLUTION

- Generate new topics
- Diversify domains
- Vary task types
- Expand coverage



Evol-Instruct (WizardLM)

 Xu, C. et al. 2023. WizardLM: Empowering Large Pre-Trained Language Models to Follow Complex Instructions.

Simple: "Write a poem" → Complex: "Write a sonnet about loss using only words starting with 's', with a twist ending"



Avoid Repetitive Data

Ensure generated examples cover diverse topics, styles, and difficulty levels.

TECHNIQUES

ROUGE-L Filtering

Remove new instructions too similar to existing ones (threshold ~0.7)

Multi-source Prompts

Draw from diverse prompt sources (UltraChat, ShareGPT, TruthfulQA...)

Multi-model Responses

Query multiple LLMs for different perspectives

Topic Balancing

Explicitly sample across domains/categories



UltraFeedback

 *UltraFeedback: Boosting Language Models with Scaled AI Feedback: 2024. <http://arxiv.org/abs/2310.01377>.*

64K prompts from 6 sources → 4 responses per prompt from different LLMs

💡 Avoid Repetitive Data

Ensure generated examples cover diverse topics, styles, and difficulty levels.

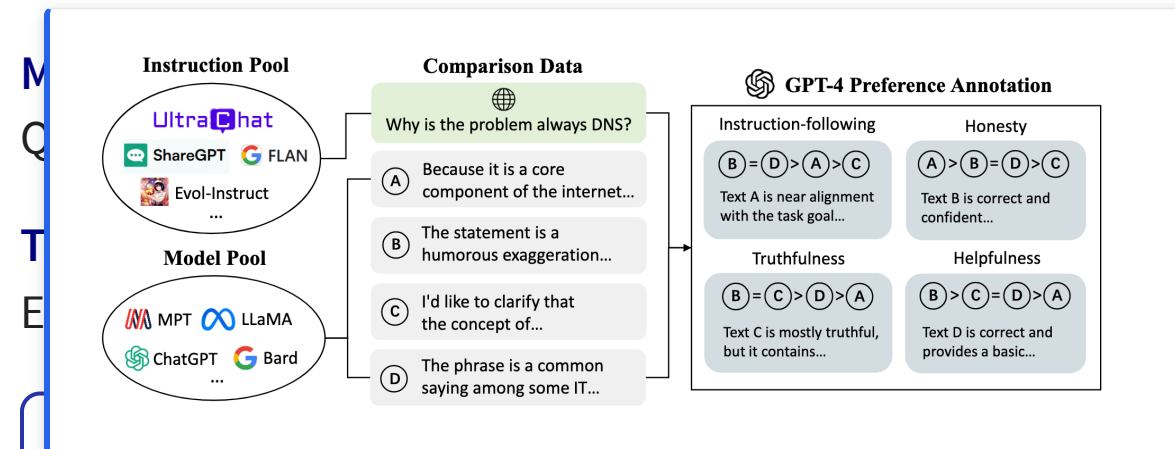
TECHNIQUES

ROUGE-L Filtering

Remove new instructions too similar to existing ones (threshold ~0.7)

Multi-source Prompts

Draw from diverse prompt sources (UltraChat, ShareGPT, TruthfulQA...)



UltraFeedback: Boosting Language Models with Scaled AI Feedback. 2024. <http://arxiv.org/abs/2310.01377>.

64K prompts from 6 sources → 4 responses per prompt from different LLMs



Remove Low-Quality Generations

Not all synthetic data is useful; filter aggressively.

FILTERING METHODS

HEURISTIC FILTERS

- Length thresholds (too short/long)
- Format validation (valid JSON, code syntax)
- Language detection
- Keyword blocklists

MODEL-BASED FILTERS

- Reward model scoring
- Perplexity thresholds
- LLM-as-judge quality rating
- Embedding deduplication



Self-Instruct Filtering

 Wang, Y. et al. 2023. *Self-Instruct: Aligning Language Models with Self-Generated Instructions*. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Filter instructions with ROUGE-L > 0.7 similarity to existing

Remove responses that are empty or too short



Expand Dataset with Variations

Create new examples by transforming existing ones.

TECHNIQUES

Paraphrasing

"Summarize this" → "Give me a brief overview of..."

Back-translation

English → French → English (introduces diversity)

Context Variation

Same task, different domain/style/persona



Back-Translation at Scale

 Edunov, S. et al. 2018. *Understanding Back-Translation at Scale*. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*.

Translate target text to source language → creates noisy but diverse training pairs

Check Correctness of Generated Data

Verify that generated examples are accurate and consistent.

TECHNIQUES

Self-Consistency

Sample multiple reasoning paths, take majority vote

Cross-Verification

Use different models to verify each other's outputs

Execution-based

For code: run tests; for math: verify computation

Self-Consistency

 *Self-Consistency Improves Chain of Thought Reasoning in Language Models: 2023.* <http://arxiv.org/abs/2203.11171>.

Generate 40 reasoning paths for same problem → select most frequent answer

+17.9% accuracy on GSM8K over greedy decoding

 Shumailov, I. et al. 2024. *AI Models Collapse When Trained on Recursively Generated Data*.

⚠ Training on Synthetic Data

Models trained on AI-generated data progressively degrade over generations.

THE PROBLEM

- Each generation loses information from the tails of the distribution
- Rare patterns and minority viewpoints disappear
- Model outputs become increasingly homogeneous
- Quality degrades exponentially over iterations

 Shumailov, I. et al. 2024. *AI Models Collapse When Trained on Recursively Generated Data*.

⚠ Training on Synthetic Data

Models trained on AI-generated data progressively degrade over generations.

THE PROBLEM

- Each generation loses information from the tails of the distribution
- Rare patterns and minority viewpoints disappear
- Model outputs become increasingly homogeneous
- Quality degrades exponentially over iterations

💡 Mitigation

- Always mix synthetic with real human data
- Preserve original training data across generations
- Monitor diversity metrics during training

CONCLUSION

- **In-Context Learning**
 - LLMs can learn from examples without parameter updates
 - Few-shot often outperforms zero-shot
 - Prompt design matters: format, order, wording

- **In-Context Learning**
 - LLMs can learn from examples without parameter updates
 - Few-shot often outperforms zero-shot
 - Prompt design matters: format, order, wording
- **Advanced Prompting**
 - Chain-of-Thought improves reasoning
 - Self-consistency provides robustness

CONCLUSION ► KEY TAKEAWAYS

- **In-Context Learning**
 - LLMs can learn from examples without parameter updates
 - Few-shot often outperforms zero-shot
 - Prompt design matters: format, order, wording
- **Advanced Prompting**
 - Chain-of-Thought improves reasoning
 - Self-consistency provides robustness
- **RL for Alignment & Agents**
 - RLHF aligns models with human preferences
 - DPO simplifies training (no reward model)
 - ReAct: reasoning + tools in autonomous loops

CONCLUSION ► KEY TAKEAWAYS

- **In-Context Learning**
 - LLMs can learn from examples without parameter updates
 - Few-shot often outperforms zero-shot
 - Prompt design matters: format, order, wording
- **Advanced Prompting**
 - Chain-of-Thought improves reasoning
 - Self-consistency provides robustness
- **RL for Alignment & Agents**
 - RLHF aligns models with human preferences
 - DPO simplifies training (no reward model)
 - ReAct: reasoning + tools in autonomous loops
- **Evaluation**
 - Traditional metrics have limitations
 - LLM-as-judge is powerful but biased
 - Contamination is a real concern for benchmarks